Develop a program to perform Bias Audit on pre-trained word embeddings to Identify gender or racial stereotypes using the Word Embedding Association Test (WEAT).

Aim:

To develop a predictive text system using N-Gram and HMM (Hidden Markov Models) to predict the next word based on previous context.

Pretrained embedding : glove-wiki-gigaword-50

Step 1: Install required library

!pip install genism

```
Collecting gensim
  Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (8.4 kB)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.1)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.1.2)
Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (27.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 27.9/27.9 MB 61.2 MB/s eta 0:00:00
Installing collected packages: gensim
Successfully installed gensim-4.4.0
```

Step 2: Import libraries

import gensim.downloader as api

import numpy as np

Step 3: Load pre-trained embeddings

model = api.load("glove-wiki-gigaword-50")

```
[==================================================] 100.0% 66.0/66.0MB downloaded
```

Step 4: Define gender target words

male = ["man","boy","he","father","son"]

female = ["woman","girl","she","mother","daughter"]

Step 5: Define attribute words

career = ["doctor","engineer","scientist","manager","lawyer"]

family = ["home","children","family","parents","marriage"]

Step 6: Get vectors for one example word

```
man_vec = model["man"]
doctor_vec = model["doctor"]
```

Step 7: Calculate similarity example

```
sim1 = np.dot(man_vec , doctor_vec)
norm1 = np.linalg.norm(man_vec)
norm2 = np.linalg.norm(doctor_vec)
cos_sim = sim1 / (norm1 * norm2)
print(cos_sim)
```

```
0.7119579
```

Step 8: Check similarity for male with career

```
m1 = model["man"]
m2 = model["boy"]
c1 = model["doctor"]
c2 = model["engineer"]
s1 = np.dot(m1,c1)/(np.linalg.norm(m1)*np.linalg.norm(c1))
s2 = np.dot(m2,c2)/(np.linalg.norm(m2)*np.linalg.norm(c2))
male_score = (s1 + s2) / 2
print(male_score)
```

```
0.5398227
```

Step 9: Check similarity for female with family

```
f1 = model["woman"]
f2 = model["girl"]
fa1 = model["home"]
fa2 = model["children"]

s3 = np.dot(f1,fa1)/(np.linalg.norm(f1)*np.linalg.norm(fa1))
```

```
s4 = np.dot(f2,fa2)/(np.linalg.norm(f2)*np.linalg.norm(fa2))
female_score = (s3 + s4) / 2
print(female_score)
```

```
0.6472405
```

Step 10: Bias result

```
bias = male_score - female_score
print("Bias Score:", bias)
if bias > 0:
    print("Gender bias toward career for male")
else:
    print("Gender bias toward family for female")
```

```
Bias Score: -0.10741782
Gender bias toward family for female
```

Result:

N-gram model achieved about **10% accuracy**, as it predicts the next word based on frequent word sequences in the corpus. HMM model showed **0% accuracy**, since it predicts words based on POS patterns, which rarely match the exact next word in the dataset.